

Meta-Labeling To Trade or Not to Trade

A second model decides whether to act on a signal



Meta-labeling splits one hard decision into two easier ones

Based on López de Prado, Advances in Financial Machine Learning, Chapter 3

01	————	Triple-barrier labels	labels that respect volatility profit-take · stop-loss · time limit
02	————	The meta-labeling idea	side first, then bet or pass primary · secondary · recall vs precision
03	————	Two examples	fewer false positives MNIST toy example · trading strategy
04	————	Position sizing	probability becomes bet size calibration · Kelly criterion
05	————	Implementation	three library functions add_vertical_barrier · get_events · get_bins

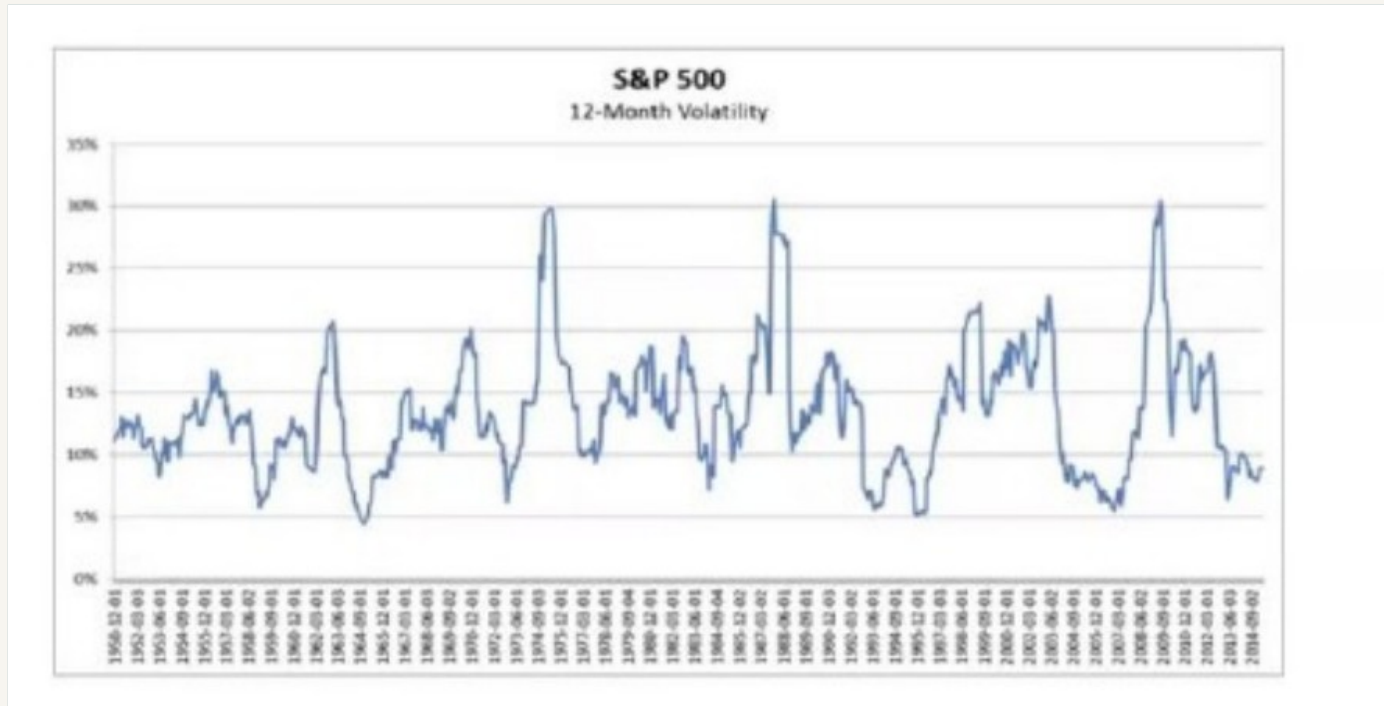
01

Triple-Barrier Labels

Label each trade by the first barrier its price path touches

Volatility moves a lot, so fixed thresholds mislabel trades

S&P 500, 12-month volatility, 1950–2014 — from about 5% to above 30%



※ A fixed $\pm 2\%$ band is wide in a calm year and narrow in a crisis — barriers should scale with volatility

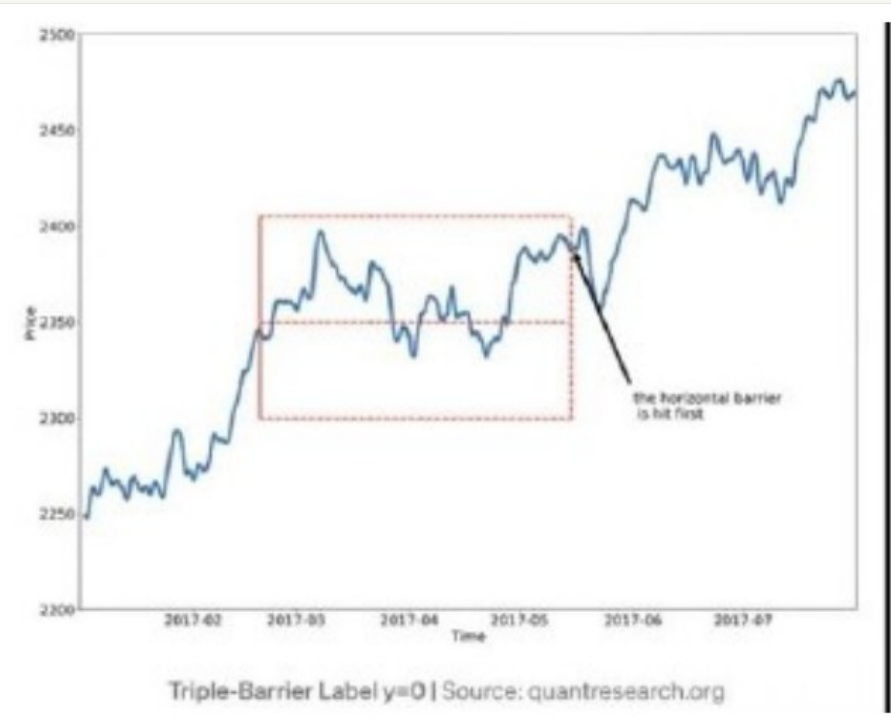
Two price barriers and one time barrier set the label

Barrier widths come from daily volatility; the first touch decides

THREE BARRIERS

- Upper: profit-take $\rightarrow +1$
- Lower: stop-loss $\rightarrow -1$
- Vertical: time limit $\rightarrow 0$

First barrier touched = the label



02

The Meta-Labeling Idea

We already have a model for the side; now decide whether to trade

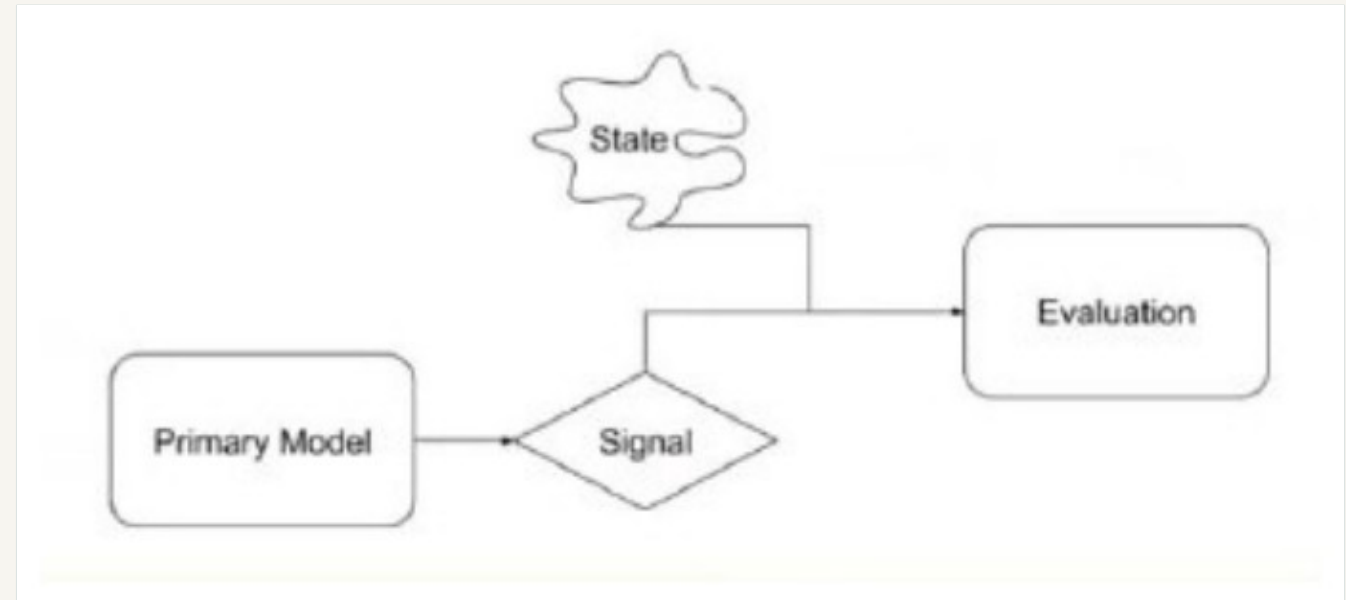
A primary model sets the side; a second model judges it

The primary model can be anything — arbitrage, technical rules, classic quant or ML

TWO STAGES

- Primary: long or short
- Secondary: right or wrong?
- Trade only if 'right'

Aim: a better recall-precision trade-off



The meta label asks one question: did the side make money?

$\hat{s}$ = side from the primary model · r = return to the first barrier touched · m = sizing function

Primary model output: the side

$$\hat{s}_i \in \{-1, +1\}$$

It never picks the direction — only bet or pass, and how much.

Meta label: the target of the secondary model

$$z_i = 1 \text{ if } \hat{s}_i r_i > 0, \quad z_i = 0 \text{ otherwise}$$

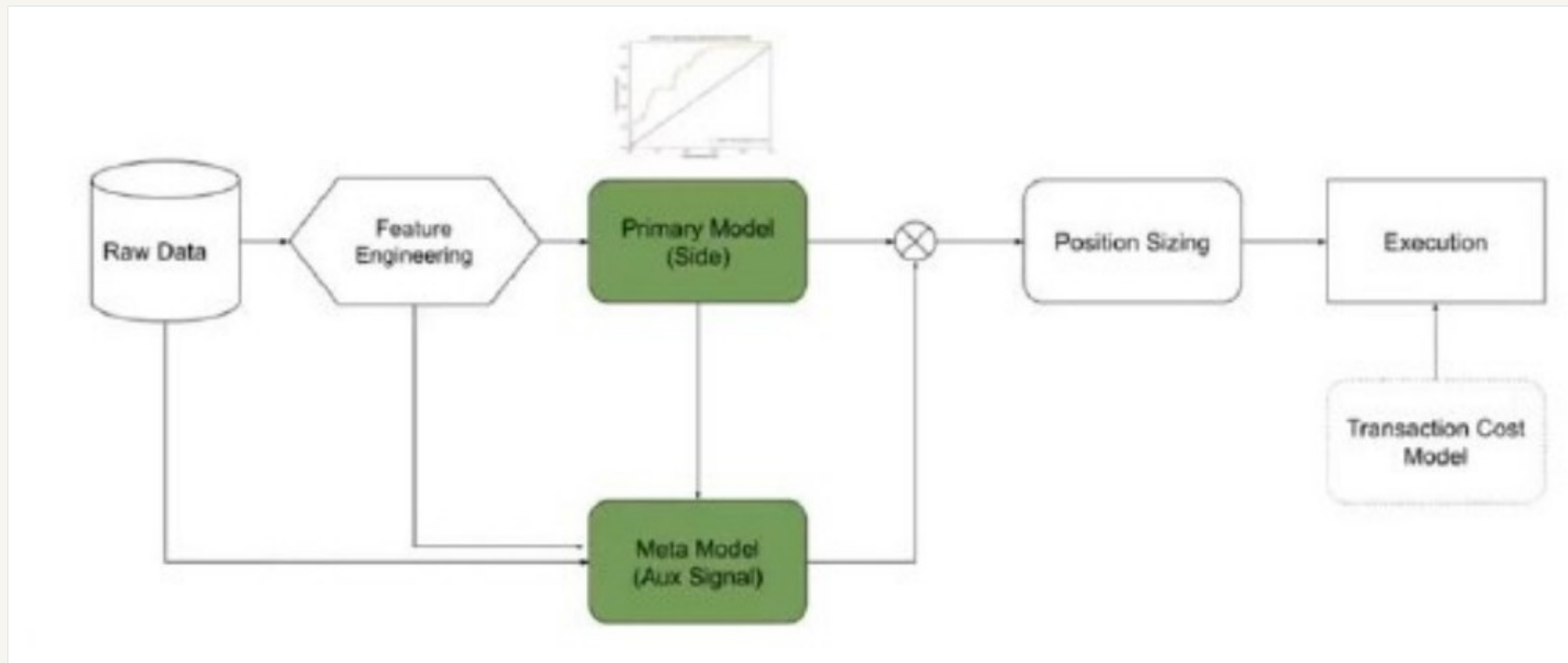
Position: side from the primary model, size from the secondary model

$$\text{position}_i = \hat{s}_i \cdot m(P(z_i = 1))$$

Meta labels are binary {0, 1}, so any classifier can learn them.

The meta model sits between the signal and position sizing

Raw data → features → primary model (side) $\otimes$ meta model (auxiliary signal) → sizing → execution



※ The meta model sees the same features plus the primary model's output; transaction costs enter at execution

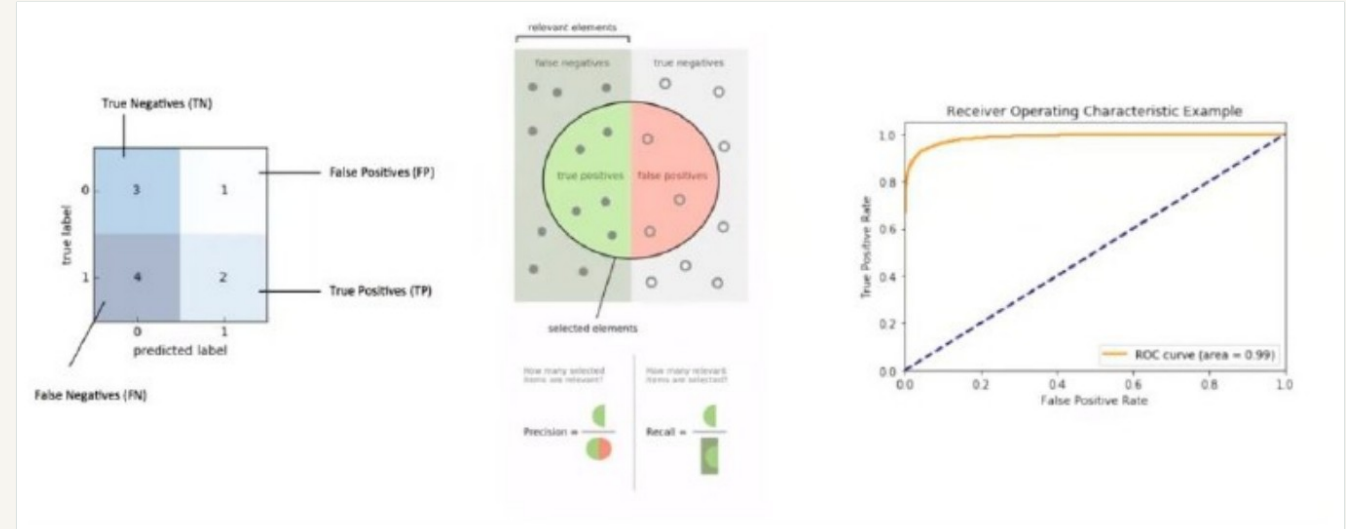
Tune the primary model for recall, then fix precision

Confusion matrix, precision and recall, and the ROC curve of the primary model

HOW IT WORKS

- Primary: catch most chances
- Low precision is fine here
- Meta: filter the positives

**Recall from stage 1,
precision from stage 2**



Recall counts the chances caught; precision counts the good trades

Positive = the primary model says 'trade'

Recall — stage 1 keeps it high

$$\text{Recall} = \frac{TP}{TP + FN}$$

Precision — stage 2 raises it

$$\text{Precision} = \frac{TP}{TP + FP}$$

PRIMARY MODEL ALONE

Many false positives

every signal is traded

WITH A META MODEL

False positives filtered

only confident signals are traded

A false positive is a losing trade; meta-labeling exists to remove them.

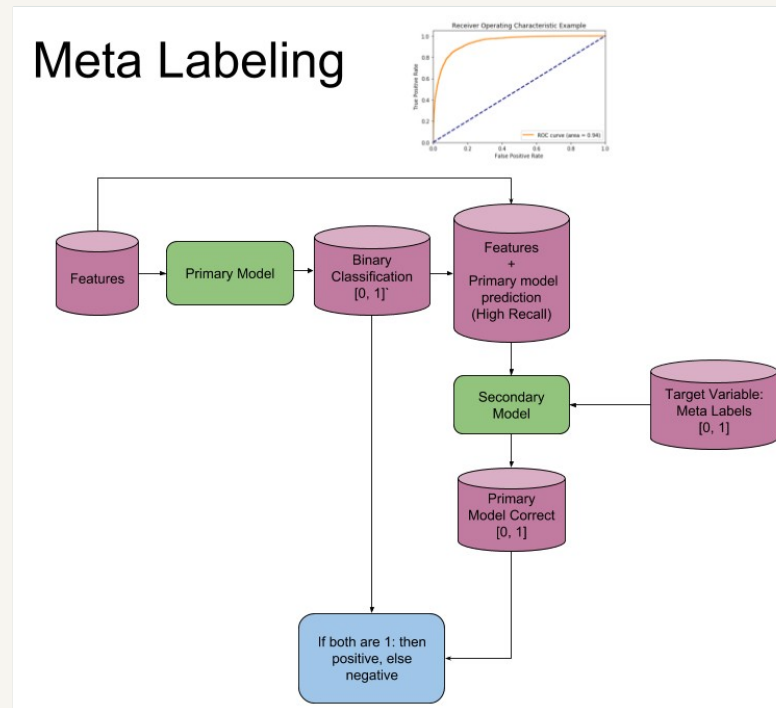
03

Two Examples

A toy classification problem, then a trading strategy

The secondary model learns when the primary model is right

Positive only if the primary model says 1 and the secondary model says it is correct



⌘ The secondary model is trained on features plus the primary prediction, with meta labels as the target

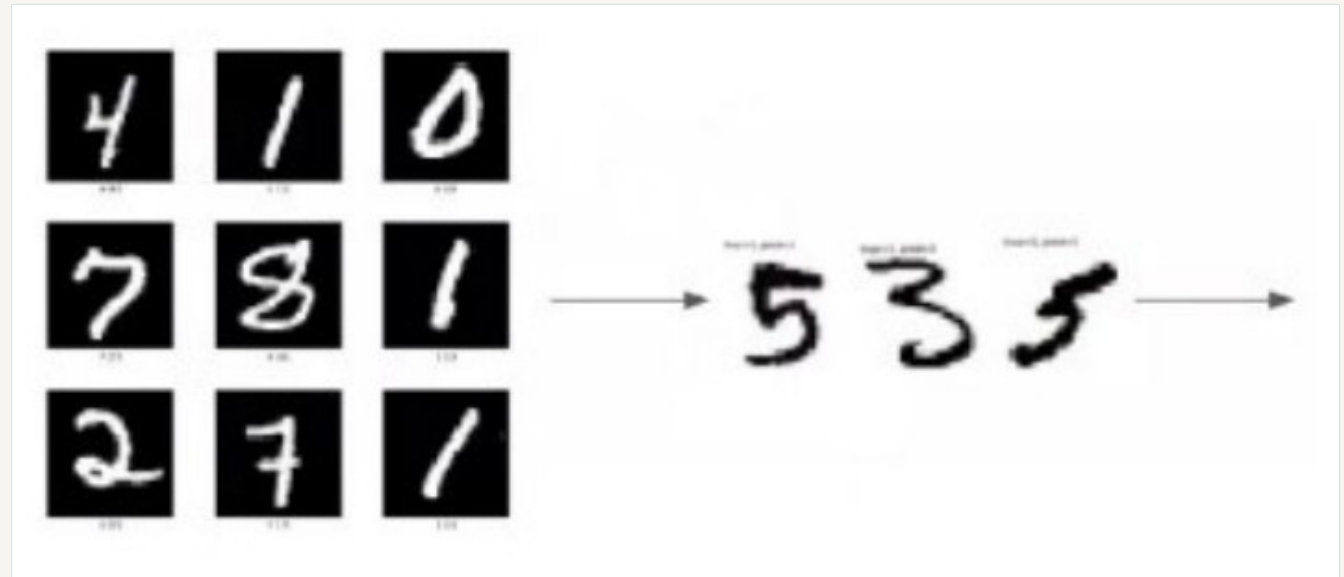
A toy example on handwritten digits shows the mechanics

MNIST images: a primary classifier tuned for high recall, then a meta model on top

SET-UP

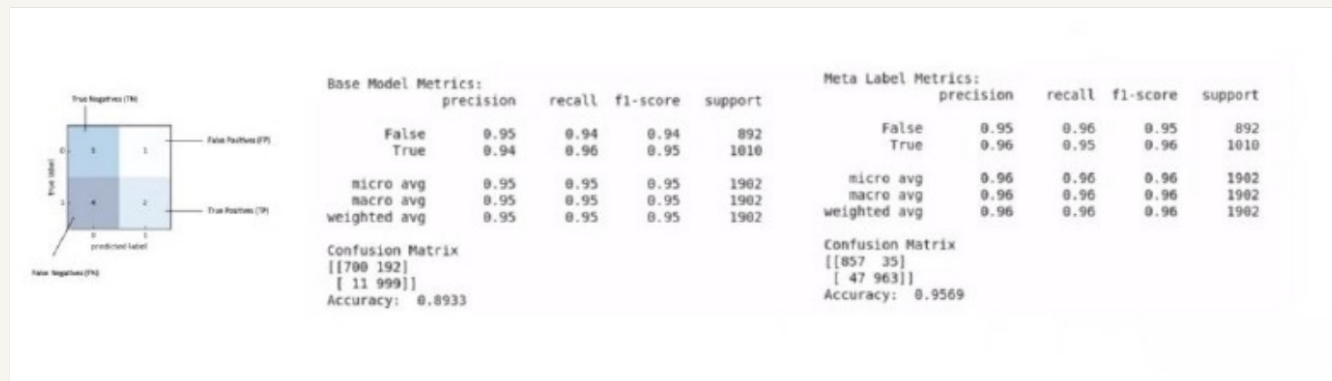
- Binary task on digit images
- Primary: high recall
- Meta: was it right?

Same recipe as for trading signals



On the digits, meta-labeling cuts false positives from 192 to 35

Confusion matrices of the primary model and the meta-labelled model, 1,902 test images



Accuracy · primary

0.893

Accuracy · meta

0.957

False positives

192 → 35

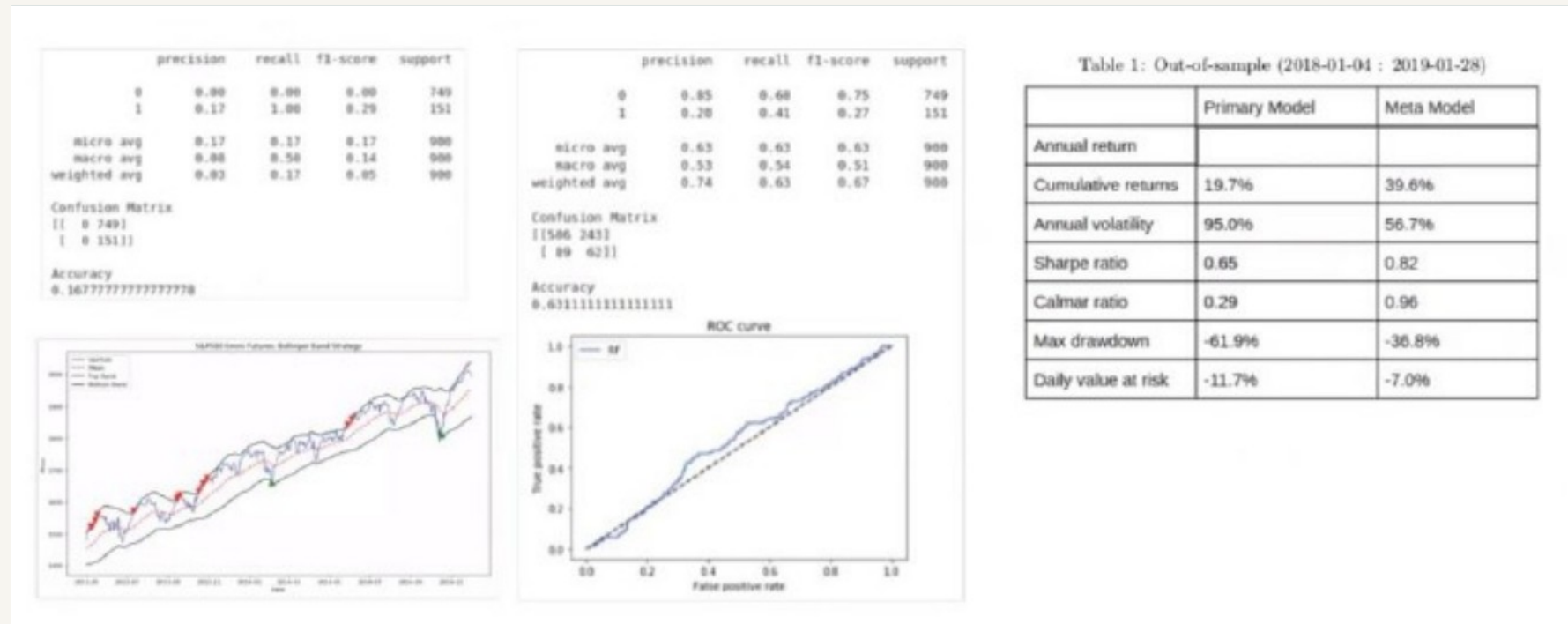
Precision (positives)

0.84 → 0.96

Recall slips from 0.99 to 0.95 — a small price for far fewer false alarms.

In the trading example, the meta model trades less and better

Primary strategy vs meta model, out of sample 2018-01-04 to 2019-01-28



✂ The primary model calls every bar positive (accuracy 0.17); the meta model passes on most of them (accuracy 0.63)

Return doubles while drawdown and risk fall

Out-of-sample table from the trading example (2018-01-04 to 2019-01-28)

	Primary model	Meta model
Cumulative return	19.7%	39.6%
Sharpe ratio	0.65	0.82
Calmar ratio	0.29	0.96
Max drawdown	−61.9%	−36.8%
Daily value at risk	−11.7%	−7.0%

Same signals, same side — only the bet-or-pass decision changed.

04

Position Sizing

Turn the secondary model's probability into a bet size

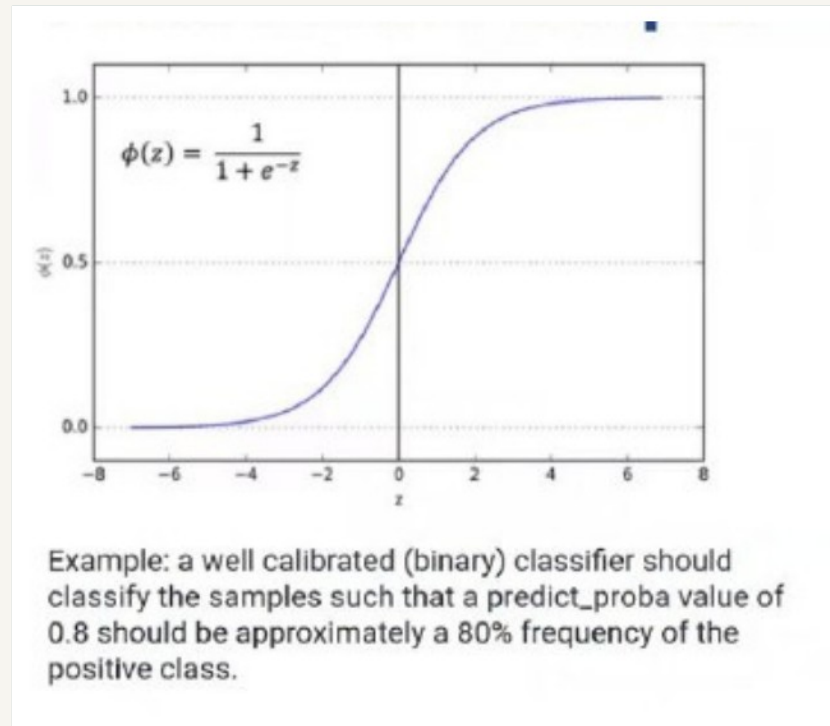
Size bets from the probability only if it is calibrated

A calibrated predict_proba of 0.8 should be right about 80% of the time

WHY CALIBRATION

- Sigmoid gives $P(z = 1)$
- Size grows with P
- Check reliability first

Uncalibrated scores give wrong sizes



The Kelly criterion sets the bet that maximises long-run growth

f^* = optimal fraction to bet · β = odds (win / loss amount) · p = win probability · $q = 1 - p$

Kelly fraction

$$f^* = \frac{\beta p - q}{\beta}$$

Investment setting

$$f^* = \frac{p}{\text{loss}} - \frac{q}{\text{profit}}$$

Even odds

$$\beta = 1 : f^* = p - q = 2p - 1$$

p comes from the meta model; loss and profit from the barriers.

A bet with no edge ($\beta p = q$) gets zero; a negative edge means do not bet.

Longer odds need a higher win rate before Kelly bets much

Thorp's example: five investment opportunities and their Kelly bets

Win probability p	Odds β	Kelly bet f^*
0.570	1 : 1	0.140
0.380	2 : 1	0.070
0.285	3 : 1	0.047
0.228	4 : 1	0.035
0.190	5 : 1	0.028

Each row has the same edge $\beta p - q = 0.14$, yet the bet shrinks as the odds lengthen.

05

Implementation

Three functions from the book's code (mlfinlab documentation)

add_vertical_barrier sets the time limit for each event

Snippet 3.4 (p. 49) — the timestamp a given number of days after each event

INPUTS → OUTPUT

- `t_events`: CUSUM events
- `close`; `num_days`, hours...
- → vertical barrier times

Passed to `get_events` as `t1`

```
add_vertical_barrier(t_events, close, num_days=0, num_hours=0, num_minutes=0, num_seconds=0)  
[source]
```

Advances in Financial Machine Learning, Snippet 3.4 page 49.

Adding a Vertical Barrier

For each index in `t_events`, it finds the timestamp of the next price bar at or immediately after a number of days `num_days`. This vertical barrier can be passed as an `optional` argument `t1` in `get_events`.

This function creates a series that has all the timestamps of when the vertical barrier would be reached.

Parameters:

- `t_events` – (pd.Series) Series of events (symmetric CUSUM filter)
- `close` – (pd.Series) Close prices
- `num_days` – (int) Number of days to add for vertical barrier
- `num_hours` – (int) Number of hours to add for vertical barrier
- `num_minutes` – (int) Number of minutes to add for vertical barrier
- `num_seconds` – (int) Number of seconds to add for vertical barrier

Returns: (pd.Series) Timestamps of vertical barriers

```
get_events(close, t_events, pt_sl, target, min_ret, num_threads, vertical_barrier_times=False,  
side_prediction=None, verbose=True) [source]
```

Advances in Financial Machine Learning, Snippet 3.6 page 50.

get_events finds the first touch, using the side if given

Snippet 3.6 (p. 50) — orchestrates the triple barrier and meta-labeling

KEY ARGUMENTS

- `pt_sl`: barrier multiples
- `target`: daily volatility
- `side_prediction`: primary side

With a side, the labels become meta labels

```
get_events(close, t_events, pt_sl, target, min_ret, num_threads, vertical_barrier_times=False,
           side_prediction=None, verbose=True) \[source\] %
```

Advances in Financial Machine Learning, Snippet 3.6 page 50.

Getting the Time of the First Touch, with Meta Labels

This function is orchestrator to meta-label the data, in conjunction with the Triple Barrier Method.

- Parameters:
- `close` – (pd.Series) Close prices
 - `t_events` – (pd.Series) of `t_events`. These are timestamps that will seed every triple barrier. These are the timestamps selected by the sampling procedures discussed in Chapter 2, Section 2.5. Eg: CUSUM Filter
 - `pt_sl` – (2 element array) Element 0, indicates the profit taking level; Element 1 is stop loss level. A non-negative float that sets the width of the two barriers. A 0 value means that the respective horizontal barrier (profit taking and/or stop loss) will be disabled.
 - `target` – (pd.Series) of values that are used (in conjunction with `pt_sl`) to determine the width of the barrier. In this program this is daily volatility series.
 - `min_ret` – (float) The minimum target return required for running a triple barrier search.
 - `num_threads` – (int) The number of threads concurrently used by the function.
 - `vertical_barrier_times` – (pd.Series) A pandas series with the timestamps of the vertical barriers. We pass a `False` when we want to disable vertical barriers.
 - `side_prediction` – (nd.Series) Side of the bet (long/short) as decided by the

get_bins turns events into labels $\{-1, 0, 1\}$ or $\{0, 1\}$

Snippet 3.7 (p. 51) — labeling for side and size

TWO CASES

- No side: label by price
- Side given: label by P&L
- $\{0, 1\}$ = bet or pass

The probability of 1 sets the size

`get_bins(triple_barrier_events, close)` [\[source\]](#)

Advances in Financial Machine Learning, Snippet 3.7, page 51.

Labeling for Side & Size with Meta Labels

Compute event's outcome (including side information, if provided). events is a DataFrame where:

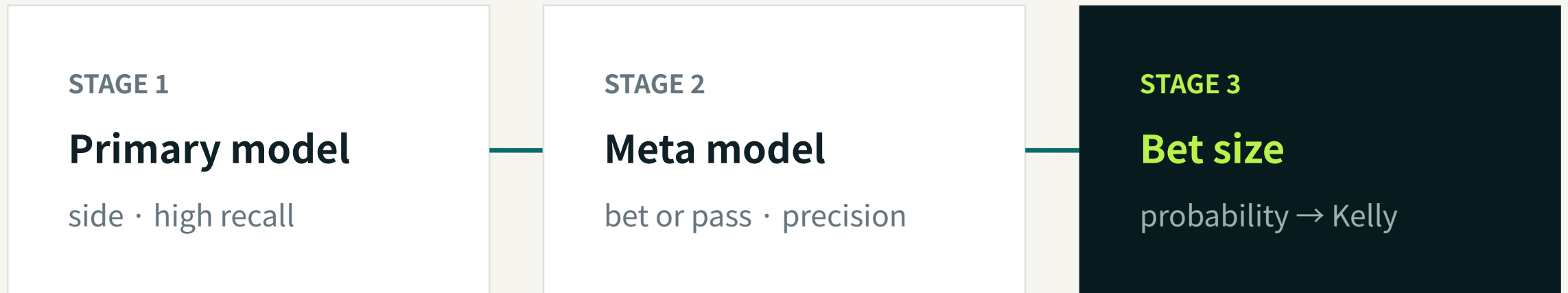
Now the possible values for labels in `out['bin']` are $\{0,1\}$, as opposed to whether to take the bet or pass, a purely binary prediction. When the predicted label the previous feasible values $\{-1,0,1\}$. The ML algorithm will be trained to decide is 1, we can use the probability of this secondary prediction to derive the size of the bet, where the side (sign) of the position has been set by the primary model.

Parameters:

- `triple_barrier_events` – (pd.DataFrame) – events.index is event's starttime – events['t1'] is event's endtime – events['trgt'] is event's target – events['side'] (optional) implies the algo's position side Case 1: ('side' not in events): bin in $\{-1,1\}$ <-label by price action Case 2: ('side' in events): bin in $\{0,1\}$ <-label by pnl (meta-labeling)

Side, bet-or-pass and size are three separate decisions

What meta-labeling adds to a trading model



Any signal — rules, quant or ML — can be improved by a meta model on top.

PRIMARY MODEL: WHICH SIDE.

META MODEL: WHETHER TO BET.

PROBABILITY: HOW MUCH.

**Fewer false positives, better-sized
bets.**